R. K. COLLEGE
BCA 2ND YER
CC5: PYTHON PROGRMMING

UNIT 1:- Introduction and Overview

INTRODUCTION

Python programming is widely used in Artificial Intelligence, Machine Learning, Neural Networks and many other advanced fields of Computer Science. Ideally, It isdesigned for rapid prototyping of complex applications. Python has interfaces withvarious Operating system calls and libraries, which are extensible to C, C++ or Java.Many large companies like NASA, Google, YouTube, Bit Torrent, etc. uses thePython programming language for the execution of their valuable projects.

To build the carrier path the skill of programming can be a fun and profitable way,but before starting the learning of this skill, one should be clear about the choice ofprogramming language. Before learning any programming language, one shouldfigure out which language suits best to the learner. As in our case the comparison of C and Python programming languages may help the learners to analyse and generate a lot of opinions about their choice of programming language. In this unit, I have tried to compile a few of them to give you a clear picture.

Python History and Versions

- Python laid its foundation in the late 1980s.
- The implementation of Python was started in December 1989 by **Guido Van Rossum** at CWI in Netherland.
- In February 1991, **Guido Van Rossum** published the code (labeled version 0.9.0) to alt.sources.
- In 1994, Python 1.0 was released with new features like lambda, map, filter, and reduce.
- Python 2.0 added new features such as list comprehensions, garbage collection systems.
- On December 3, 2008, Python 3.0 (also called "Py3K") was released. It was designed to rectify the fundamental flaw of the language.
- *ABC programming language* is said to be the predecessor of Python language, which was capable of Exception Handling and interfacing with the Amoeba Operating System.
- The following programming languages influence Python:
    - ABC language.
    - Modula-3

Why the Name Python?

There is a fact behind choosing the name Python. **Guido van Rossum** was reading the script of a popular BBC comedy series "**Monty Python's Flying Circus**". It was late on-air 1970s.

Van Rossum wanted to select a name which unique, sort, and little-bit mysterious. So he decided to select naming Python after the **"Monty Python's Flying Circus"** for their newly created programming language.

The comedy series was creative and well random. It talks about everything. Thus it is slow and unpredictable, which made it very interesting.

Python is also versatile and widely used in every technical field, such as Machine Learning, Artificial Intelligence, Web Development, Mobile Application, Desktop Application, Scientific Calculation, etc.

COMPARISON

Metrics	Python	C
Introduction	It is a high-level, general-purpose & interpreted programming language.	is general-purpose procedural programming language.
Speed	Being Interpreted programming language its execution speed is slower then that of the compiled programming language (i.e. C).	Being compiled programming language its execution speed is faster then that of the interpreted programming language (i.e. Python).
Usage	Number of lines of code written in Python is quite less in comparison to C	Program syntax of C is quite complicated in comparison to Python.
Declaration of variables	Declaration of Variable type is not required, they are un-typed and a given variable can be stuck different types of values at different instances during the execution of any python program.	In C, declaration of variable type is must, and it is done at the time of its creation, and only values of that declared type must be assigned to the variables.
Error Debugging	Error debugging is simple, as it takes only one instruction at a time. Compilation and execution is performed simultaneously. Errors are instantly shown, and execution stops at that instruction only.	Being compiler dependent language, error debugging is difficult in C i.e. it takes the entire source code, compiles it and finally all errors are shown.
Function renaming mechanism	the same function can be used by two different names i.e. it supports the mechanism of function renaming	Function renaming mechanism is not supported by C i.e. the same function cannot be used by two different names.
Complexity	Syntax of Python programs is Quite simple, easy to read, write and learn	The syntax of a C program is comparatively harder than the syntax of Python.
Applications	Python is a General-Purpose programming language can be	C is generally used for hardware related applications.

	used for Microcontrollers also.	
Built-in Functions	Library of built-in functions is quite large in Python.	Library of built-in functions is quite limited in C
Implementing Data Structures	Gives ease of implementing data structures with built-in insert, append functions	Explicit implementation of functions are requited for the implementation of data structures
Pointers	Pointers functionality is not available in Python.	Pointers are frequently used in C.

PYTHON COMMENTS

Comments in Python are the lines in the code that are ignored by the interpreter during the execution of the program. Comments enhance the readability of the code and help the programmers to understand the code very carefully. There are three types of comments in Python –

- Single line Comments
- Multiline Comments
- Docstring Comments

SINGLE-LINE COMMENTS

Single-line remarks in Python have shown to be effective for providing quick descriptions for parameters, function definitions, and expressions. A single-line comment of Python is the one that has a hashtag # at the beginning of it and continues until the finish of the line. If the comment continues to the next line, add a hashtag to the subsequent line and resume the conversation. Consider the accompanying code snippet, which shows how to use a single line comment:

Code

```
# This code is to show an example of a single-line comment
print( 'This statement does not have a hashtag before it' )
```

Output:

```
This statement does not have a hashtag before it
```

MULTI-LINE COMMENTS

Python does not provide the facility for multi-line comments. However, there are indeed many ways to create multi-line comments.

With Multiple Hashtags (#)

In Python, we may use hashtags (#) multiple times to construct multiple lines of comments. Every line with a (#) before it will be regarded as a single-line comment.

Code

```
# it is a
# comment
# extending to multiple lines
```

In this case, each line is considered a comment, and they are all omitted.

Using String Literals

Because Python overlooks string expressions that aren't allocated to a variable, we can utilize them as comments.

Code

```
1. 'it is a comment extending to multiple lines'
```

We can observe that on running this code, there will be no output; thus, we utilize the strings inside triple quotes(""") as multi-line comments.

- **PYTHON DOCSTRING**

The strings enclosed in triple quotes that come immediately after the defined function are called Python docstring. It's designed to link documentation developed for Python modules, methods, classes, and functions together. It's placed just beneath the function, module, or class to explain what they perform. The docstring is then readily accessible in Python using the __doc__ attribute.

Code

```
1. # Code to show how we use docstrings in Python
2.
3. def add(x, y):
4.     """This function adds the values of x and y"""
5.     return x + y
6.
7. # Displaying the docstring of the add function
8. print( add.__doc__ )
```

Output:

```
This function adds the values of x and y
```

<u>USING PYTHON AS A CALCULATOR</u>

Numbers

The interpreter acts as a simple calculator: you can type an expression at it and it will write the value. Expression syntax is straightforward: the operators +, -, * and / work just like in most other languages (for
example, Pascal or C); parentheses (()) can be used for grouping. For example:

```
>>>2 + 2
4
>>>50 - 5*6
20
>>>(50 - 5*6) / 4
5.0
```

```
>>>8 / 5 # division always returns a floating point number
1.6
```

Division (/) always returns a float. To do *floor division* and get an integer result (discarding any fractional
result) you can use the // operator; to calculate the remainder you can use %:

```
>>>17 / 3 # classic division returns a float
5.666666666666667
>>>
>>>17 // 3 # floor division discards the fractional part
5
>>>17 % 3 # the % operator returns the remainder of the division
2
>>>5 * 3 + 2 # result * divisor + remainder
17
```

With Python, it is possible to use the ** operator to calculate powers1:

```
>>>5 ** 2 # 5 squared
25
>>>2 ** 7 # 2 to the power of 7
128
```

The equal sign (=) is used to assign a value to a variable. Afterwards, no result is displayed before the next
interactive prompt:

```
>>>width = 20
>>>height = 5 * 9
>>>width * height
900
```

when you are using Python as a desk calculator, it is somewhat easier to continue calculations, for example:

```
>>>tax = 12.5 / 100
>>>price = 100.50
>>>price * tax
12.5625
>>>price + _
113.0625
>>>round(_, 2)
113.06
```

STRINGS

Besides numbers, Python can also manipulate strings, which can be expressed in several ways. They can be enclosed in single quotes ('...') or double quotes ("...") with the same result2. \ can be used to escape quotes:

```
>>>'rahmat' # single quotes
'rahmat'
```

```
>>>'doesn\'t' # use \' to escape the single quote...
"doesn't"
>>>"doesn't" # ...or use double quotes instead
"doesn't"
>>>'"Yes," they said.'
'"Yes," they said.'
>>>"\"Yes\" they said."
'"Yes," they said.'
>>>'"Isn\'t," they said.'
'"Isn\'t," they said.'

>>>s = 'First line.\nSecond line.' # \n means newline
>>>s # without print(), \n is included in the output
'First line.\nSecond line.'
>>>print(s) # with print(), \n produces a new line
First line.
Second line.
```

Two or more *string literals* (i.e. the ones enclosed between quotes) next to each other are automatically concatenated.

```
>>>'Py' 'thon'
'Python'
```

This feature is particularly useful when you want to break long strings:

```
>>>text = ('Put several strings within parentheses '
... 'to have them joined together.')
>>>text
'Put several strings within parentheses to have them joined together.'
```

Strings can be *indexed* (subscripted), with the first character having index 0. There is no separate character type; a character is simply a string of size one:

```
>>>word = 'Python'
>>>word[0] # character in position 0
'P'
>>>word[5] # character in position 5
'n'
```

Note that since -0 is the same as 0, negative indices start from -1.In addition to indexing, *slicing* is also supported. While indexing is used to obtain individual characters, *slicing* allows you to obtain substring:

```
>>>word[0:2] # characters from position 0 (included) to 2 (excluded)
'Py'
>>>word[2:5] # characters from position 2 (included) to 5 (excluded)
'tho'
```

Note how the start is always included, and the end always excluded. This makes sure that s[:i] + s[i:] is always equal to s:

```
>>>word[:2] + word[2:]
'Python'
```

```
>>>word[:4] + word[4:]
'Python'
```

Slice indices have useful defaults; an omitted first index defaults to zero, an omitted second index defaults
to the size of the string being sliced.

```
>>>word[:2] # character from the beginning to position 2 (excluded)
'Py'
>>>word[4:] # characters from position 4 (included) to the end
'on'
>>>word[-2:] # characters from the second-last (included) to the end
'on'
```

The built-in function len() returns the length of a string:

```
>>>s = 'supercalifragilisticexpialidocious'
>>>len(s)
34
```

VARIABLES

A**variable** is a named placeholder to hold any type of data which the program can use to assign and modify during the course of execution. Python is a Dynamically Typed Language. There is no need to declare a variable explicitly by specifying whether the variable is an integer or a float or any other type.
To define a new variable in Python, we simply assign a value to a name. Variable names can be arbitrarily long. They can contain both letters and numbers, but they cannot start with a number. It is legal to use uppercase letter; but it is a good idea to begin variable names with a lowercase letter.
Variable names are **case-sensitive**. E.g., SANDIPUNIVERSITY and sandipuniversity are differentvariables.

Variables are the example of identifiers. An Identifier is used to identify the literals used in the program. The rules to name an identifier are given below.

- The first character of the variable must be an alphabet or underscore (_).
- All the characters except the first character may be an alphabet of lower-case(a-z), upper-case (A-Z), underscore, or digit (0-9).
- Identifier name must not contain any white-space, or special character (!, @, #, %, ^, &, *).
- Identifier name must not be similar to any keyword defined in the language.
- Identifier names are case sensitive; for example, my name, and MyName is not the same.
- Examples of valid identifiers: a123, _n, n_9, etc.
- Examples of invalid identifiers: 1a, n%4, n 9, etc.

The **underscore character**can appear in a name. it is often used in names with multiple words, such as my_name or marks_in_maths. Variable names can start with an underscore character, but we generally avoid doing this unless we are writing library code for others to use.

The general format for assigning values to variables is as follows:

variable_name = expression

The equal sign (=) also known as simple assignment operator is used to assign values to variables.

number = 100#integer type value is assigned to a variable *number*
name = "Sunita" #string type value is assigned to variable *name*

Python allows you to assign a single value to several variables simultaneously. E.g.**x = y = z =5** #an integer value is assigned to variables x, y, z simultaneously.

Q. Which of the following is a valid variable name in Python?
a) do it
b) do+1
c) 1do
d) All of the above
e) None of the above
Q. What is the value of 'a' here?
a = 25
b = 35
a = b
a) 25
b) 35
c) It will print error
d) None of the above

Q) Which of the following is true for variable names in Python?
a) unlimited length
b) variable names are not case sensitive
c) underscore and ampersand are the only two special characters allowed
d) none of the mentioned
Q. What error occurs when you excecute the following Python Code snippet?
apple = mango
a) SyntaxError
b) NameError
c) ValueError
d) TypeError

OPERATORS

Operators are special symbols that represent computations like addition and multiplication. The values the operator is applied to are called operands.

Python language supports a wide range of operators. They are:

1. Arithmetic Operators
2. Assignment Operators
3. Relational Operators
4. Logical Operators
5. Bitwise Operators

1. Arithmetic Operators

Operator	**Operator Name**	**Description**	**Example (try in lab)**

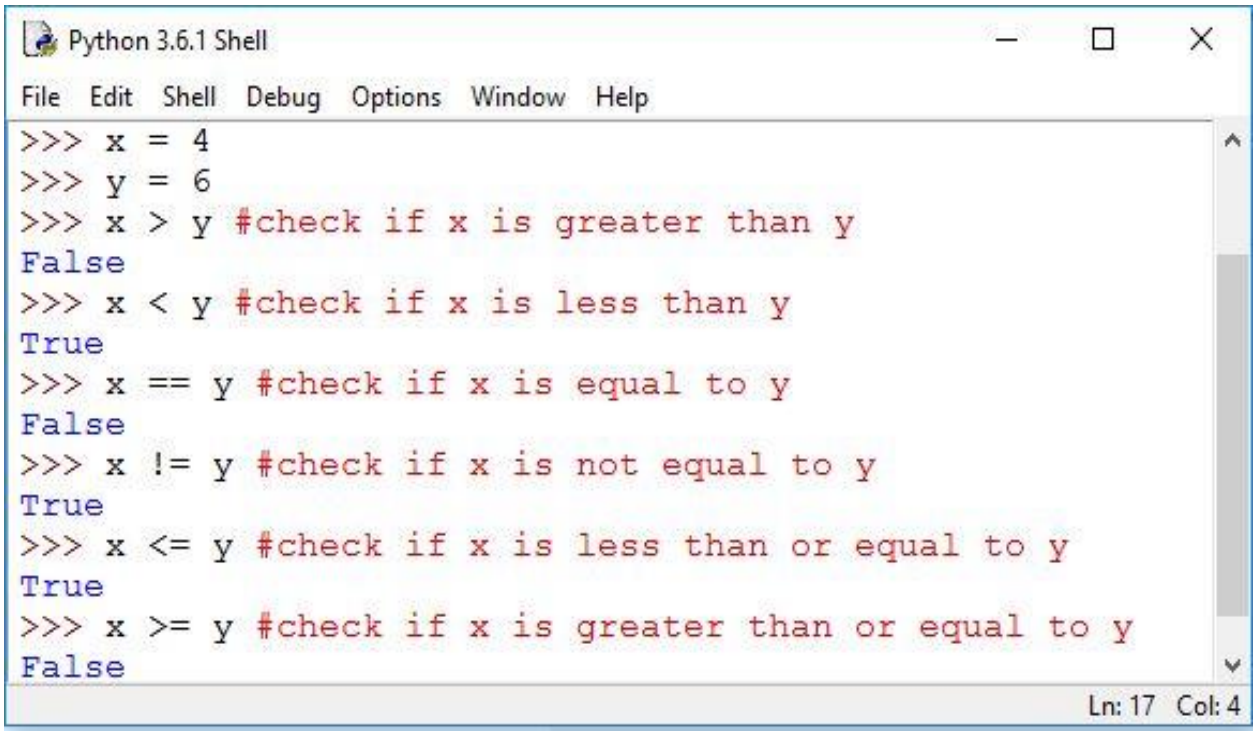
			n1 = 5, n2 = 6
+	Addition operator	Adds two operands, producing their sum	**n1 + n2** **11**
-	Subtraction Operator	Subtracts the two operands, producing their difference	**n1 - n2** **-1**
*	Multiplication Operator	Produces the product of the operands	**n1 * n2** **30**
/	Division operator	Produces the quotient of its operands where the left operand is the dividend and the right operand is the divisor	**n1 / n2** **0.833333**
%	Modulus operator	Divides left hand operand by the right hand operand and returns a remainder	**n1 % n2** **5**
**	Exponent operator	Performs exponential (power) calculation on operators	**n1 ** n2** **15625**
//	Floor division Operator	Returns the integral part of the quotient	**n1 // n2** **0**

2. Assignment Operators

Operator	**Operator Name**	**Description**	**Example (try in lab) ('~' stand for is equivalent to)**
=	Assignment	Assigns values from right side operands to left side operand	m = n1 + n2 In[]: **m = n1 + n2** In[]: **m** Out[]: **11**
+=	Addition Assignment	Adds the value of right operand to the left operand and assigns the result to left operand	m += n1 ~ m = m + n1 In[]: **m += n1** In[]: **m** Out[]: **16**
-=	Subtraction Assignment	Subtracts the value of right operand from the left operand and assigns the result to left operand	m -= n1 ~ m = m – n1 In[]: **m -= n1** In[]: **m** Out[]: **11**
*=	Multiplication Assignment	Multiplies the value of right operand with the left operand and assigns the result to left operand	m *= n1 ~ m = m* n In[]: **m *= n1** In[]: **m** Out[]: **55**

/=	Division Assignment	Divides the value of right operand with the left operand and assigns the result to left operand	m /= n1 ~ m = m / n1 In[]: **m /= n1** In[]: **m** Out[]: **11.0**
**=	Exponentiation Assignment	Evaluates to the result of raising the first operand to the power of the second operand	m ** n1 ~ m = m ** n1 In[]: **m **= n1** In[]: **m** Out[]: **161051.0**

3. Relational Operators



n1 = 10, n2 = 0,
s1 = "Rahmat",
s2="Raj"
== Equals to If values of two operands are equal, then the conditionis True, otherwise it is False.

In[]: **n1 ==n2**
Out[]: **False**

In[] : **s1 ==s2**
Out[]: **False**

!= Not equal to If values of two operands are not equal, then condition

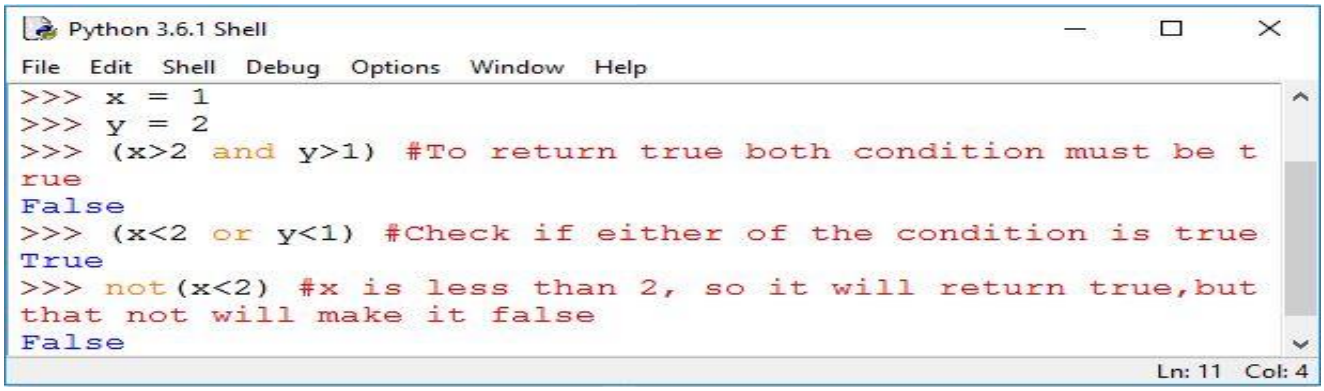
is True, otherwise it is False.
In[] : **n1 != n2**
Out[]: **True**

In[]: **s1 != s2**
Out[]: **False**

Logical Operators

There are three Python logical operators: and or and not.

Operator	Meaning
And	(x and y) will return true if x and y both are true
Or	(x or y) will return true if either of x or y is true
Not	(not x) will return true if x is false or will return false if x is true. not reverses or complements the operands



Python Bitwise Operators

Bitwise operators take binary digits as operands and perform bit by bit operations.

Following is the list of bitwise operators supported in Python.

let's assume:

- a = 5 = 0101 (in binary)
- b = 7 = 0111 (in binary)

Now if we were to use bitwise operator AND (&), it would generate following output.

a&b = 0101 & 0111 = 0101

Operator	Example	Operation
& (Binary AND)	a & b	0101 & 0111 = 0101
| (Binary OR)	a | b	0101 | 0111 = 0111
^ (Binary XOR)	a ^ b	0101 ^ 0111 = 0010
~ (Binary NOT)	~ a	~5 = -6 (11111010)
>> (Binary right shift)	a >> 1	0101>>1 = 0010
<< (Binary left shift)	b << 1	0111<<1 = 1110

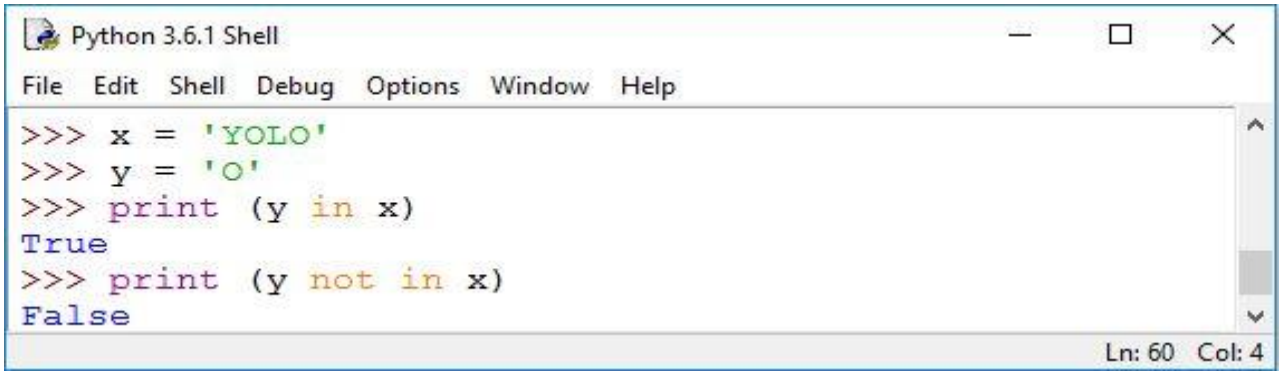
Python Membership Operators

Python membership operator is used to check whether a variable is a member of a sequence such as String, List, Tuple and Dictionary.

Note: In the Python dictionary, we can check the membership of key only, not the value.

Here are the membership operators in Python.

Operator	Example	Description
In	x in y	True if x is the member of y
not in	x not in y	True if x is not the member of y



Precedence of Python Operators

Here is the list of Python Operators in descending order, listing from higher precedence to lower precedence.

Operator	Description
( )	Parentheses
**	Exponent (raise to the power)
, -, ~	Unary plus, Unary minus and Bitwise NOT
*, /, %, //	Multiplication, Division, Modulus and Floor Division
+, –	Addition and Subtraction
>>, <<	Bitwise Right Shift and Bitwise Left Shift
&	Bitwise AND
^, |	Bitwise XOR and OR
<=, <, >, >=	Comparison Operators
==, !=	Equality Operators

=, %=, /=, //=, -=, +=, *=, **=	Assignment Operators
is, is not	Identity Operators
in, not in	Membership Operators
not, or, and	Logical Operators

- PYTHON LIST

A list in Python is used to store the sequence of various types of data. A list can be defined as a collection of values or items of different types. Python lists are mutable type which implies that we may modify its element after it has been formed. The items in the list are separated with the comma (,) and enclosed with the square brackets [].

Although Python has six data types that may hold sequences, the list is the most popular and dependable form. The collection of data is stored in a list, a sequence data type. Similar sequence data formats are Tuples and String.

Python lists are identical to dynamically scaled arrays that are specified in other languages, such as Java's ArrayList and C++'s vector. A list is a group of items that are denoted by the symbol [] and subdivided by commas.

List Declaration

Code

```
# a simple list
list1 = [1, 2, "Python", "Program", 15.9]
list2 = ["Amy", "Ryan", "Henry", "Emma"]

# printing the list
print(list1)
print(list2)

# printing the type of list
print(type(list1))
print(type(list2))
```

Output:

```
[1, 2, 'Python', 'Program', 15.9]
['Amy', 'Ryan', 'Henry', 'Emma']
< class ' list ' >
< class ' list ' >
```

Characteristics of Lists

The list has the following characteristics:

- The lists are ordered.
- The element of the list can access by index.
- The lists are the mutable type.
- The lists are mutable types.
- A list can store the number of various elements.

PYTHON TUPLES

A Python Tuple is a group of items that are separated by commas. The indexing, nested objects, and repetitions of a tuple are somewhat like those of a list, however unlike a list, a tuple is immutable.

The distinction between the two is that while we can edit the contents of a list, we cannot alter the elements of a tuple once they have been assigned.

Example

1. ("Suzuki", "Audi", "BMW"," Skoda ") is a tuple.

Features of Python Tuple

- Tuples are an immutable data type, which means that once they have been generated, their elements cannot be changed.
- Since tuples are ordered sequences, each element has a specific order that will never change.

Creating of Tuple:

Code:

```
# Python program to show how to create a tuple
# Creating an empty tuple
empty_tuple = ()
print("Empty tuple: ", empty_tuple)

# Creating tuple having integers
int_tuple = (4, 6, 8, 10, 12, 14)
print("Tuple with integers: ", int_tuple)
```

```
# Creating a tuple having objects of different data types
mixed_tuple = (4, "Python", 9.3)
print("Tuple with different data types: ", mixed_tuple)

# Creating a nested tuple
nested_tuple = ("Python", {4: 5, 6: 2, 8:2}, (5, 3, 5, 6))
print("A nested tuple: ", nested_tuple)
```

Output:

```
Empty tuple:  ()
Tuple with integers:  (4, 6, 8, 10, 12, 14)
Tuple with different data types:  (4, 'Python', 9.3)
A nested tuple:  ('Python', {4: 5, 6: 2, 8: 2}, (5, 3, 5, 6))
```

PYTHON DICTIONARY

An effective data structure for storing data in Python is dictionaries, in which can simulate the real-life data arrangement where some specific value exists for some particular key.

- Python Dictionary is used to store the data in a key-value pair format.
- It is the mutable data-structure.
- The elements Keys and values is employed to create the dictionary.
- Keys must consist of just one element.
- Value can be any type such as list, tuple, integer, etc.

In other words, we can say that a dictionary is the collection of key-value pairs where the value can be of any Python object. In contrast, the keys are the immutable Python object, i.e., Numbers, string, or tuple. Dictionary entries are ordered as of Python version 3.7. In Python 3.6 and before, dictionaries are generally unordered.

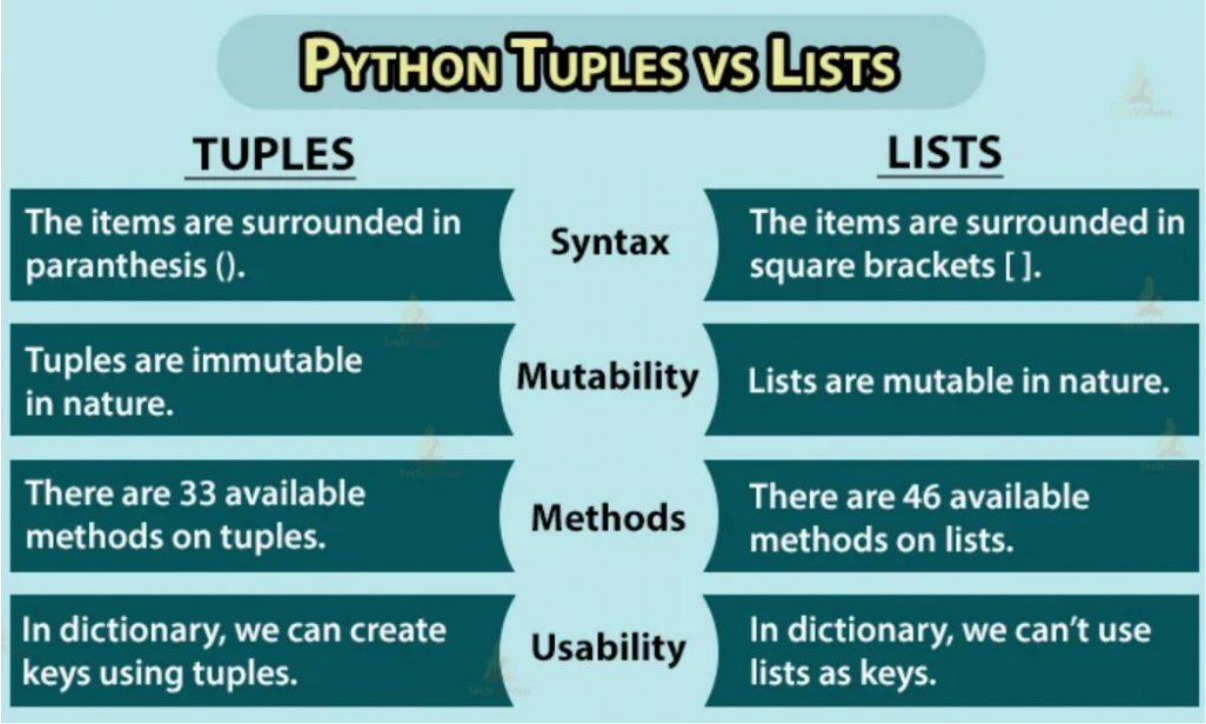
Creating the Dictionary

Code

```
Employee = {"Name": "John", "Age": 29, "salary":25000,"Company":"GOOGLE"}
print(type(Employee))
print("printing Employee data .... ")
print(Employee)
```

Output

```
<class 'dict'>
printing Employee data ....
{'Name': 'John', 'Age': 29, 'salary': 25000, 'Company': 'GOOGLE'}
```



CONTROL FLOW STATEMENTS

1. Decision Control Flow Statements: Depending on whether a condition is True or False, the decision structure may skip the execution of an entire block of statements or even execute one block of statements instead of other (if, if…else and if…elif…else).

2.Loop Control Flow Statements: This is a control structure that allows the execution of a block of statements multiple times until a loop termination condition is met (for loop and while loop). Loop Control Flow statements are also called Repetition statements of Iteration Statements.

The syntax for if statement is:

if Boolean_Expression :
statement (s)

```
>>>x = int(input("Please enter an integer: "))
Please enter an integer: 42
>>>if x <0:
... x = 0
... print('Negative changed to zero')
... elifx== 0:
... print('Zero')
...elifx== 1:
... print('Single')
... else:
... print('More')
...
```

More

There can be zero or more elif parts, and the else part is optional. The keyword 'elif' is short for 'else if', and is useful to avoid excessive indentation. An if … elif … elif … sequence is a substitute for the switch or case statements found in other languages.

```
In [7]: num=int(input("Enter a number : "))
        if num==1:
            print("Monday")
        elif num==2:
            print("Tuesday")
        elif num==3:
            print("Wednesday")
        elif num==4:
            print("Thursday")
        elif num==5:
            print("Friday")
        elif num==6:
            print("Saturday")
        elif num==7:
            print("Sunday")
        else:
            print("Please enter a number between 1 and 7")
```

Nested if – (If within if)

```
year = int(input('Enter a year : '))
if year % 4 == 0:
  if year % 100 == 0 :
    if year % 400 == 0 :
      print(f'{year} is a Leap Year')
    else :
      printf(f'{year} is not a Leap Year')
  else :
    print(f'{year} is a Leap Year')
else :
  print(f'{year} is not a Leap Year')
```

```
Enter a year : 2014
2014 is not a Leap Year
```

for Statements

A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

```
fruits = ["apple", "grapes", "mango"]
for x in fruits:
  print(x)
```

```
for x in "rahmat":
 print(x)
output:  r
         a
         h
         m
         a
         t
```

```
>>># Measure some strings:
... words = ['cat', 'window', 'Sandipuniversity']
>>>for w in words:
... print(w, len(w))
...
cat 3
window 6
Sandipuniversity 16
```

Nested Loop

```
In [3]: colors = ['yellow', 'black', 'green', 'purple']
        fruits = ['mango',"banana", "apple"]
        for x in colors:
            for y in fruits:
                print(x,y)

        yellow mango
        yellow banana
        yellow apple
        black mango
        black banana
        black apple
        green mango
        green banana
        green apple
        purple mango
        purple banana
        purple apple
```

4.3 The range () Function

If you do need to iterate over a sequence of numbers, the built-in function range() comes in handy. It generates arithmetic progressions:

```
>>>foriin range (5):
... print(i)
...
0
1
2
3
4
```

To iterate over the indices of a sequence, you can combine range() and len() as follows:

```
>>>a = ['Mary', 'had', 'a', 'little', 'lamb']
>>>for iin range(len(a)):
... print(i, a[i])
...
0 Mary
1 had
2 a
3 little
4 lamb
```

The continue and break Statements

The break statement, like in C, breaks out of the innermost enclosing for or while loop.
Loop statements may have an else clause; it is executed when the loop terminates through exhaustion of the list (with for) or when the condition becomes false (with while), but not when the loop is terminated by a break statement. This is exemplified by the following loop, which searches for prime numbers:

```
>>>for n in range(2, 10):
... for x in range(2, n):
... if n % x == 0:
...print(n, 'equals', x, '*', n//x)
...break
...else:
...# loop fell through without finding a factor
... print(n, 'is a prime number')
...

2 is a prime number
3 is a prime number
4 equals 2 * 2
5 is a prime number
6 equals 2 * 3
7 is a prime number
8 equals 2 * 4
9 equals 3 * 3
```

```
count = 0
while True :
    count += 1
    if count > 5 :
        break
    print (count)
```

```
1
2
3
4
5
```

```
fruits = ["apple", "orange", "mango"]
for x in fruits:
  print(x)
  if x == "banana":
    break
```

```
>>>for n in range(2, 10):
... for x in range(2, n):
... if n % x == 0:
... print(n, 'equals', x, '*', n//x)
... break
... else:
... # loop fell through without finding a factor
... print(n, 'is a prime number')
...
2 is a prime number
3 is a prime number
4 equals 2 * 2
5 is a prime number
6 equals 2 * 3
7 is a prime number
8 equals 2 * 4
9 equals 3 * 3
```

The continue statement, also borrowed from C, continues with the next iteration of the loop:

```
>>>for numinrange(2, 10):
..... if num% 2 == 0:
.....print("Found an even number", num)
.....continue
.....print("Found a number", num)

Found an even number 2
```

Defining Functions

We can create a function that writes the Fibonacci series to an arbitrary boundary:

```
>>>deffib(n): # write Fibonacci series up to n
... """Print a Fibonacci series up to n."""
... a, b = 0, 1
... while a <n:
... print (a, end=' ')
... a, b = b, a+b
... print ()
...
>>># Now call the function we just defined:
... fib (2000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597
```